

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..26

OVERVIEW

 Copy as Markdown

Overview

Serverpod comes with a built-in web server that runs beside the API server. It serves anything that speaks plain HTTP: REST APIs and webhooks, static files, server-rendered HTML with templates or [Jaspr](#), and single-page apps including Flutter web. Web requests get the same [Session](#) your endpoint methods receive, with full access to your database and business logic. The web server is built on the [Relic](#) framework, and its routing engine, middleware system, and typed headers are available directly.

The web server and the API server answer different callers. [Endpoints](#) are the typed methods your own app calls through the generated client. Web server **routes** serve everyone else: browsers, webhooks, third-party services, and anything that needs a URL. If you are building a feature for your Flutter app, write an endpoint. If something outside your app needs to reach your server over HTTP, write a route.

Your first route

New Serverpod projects set up the web server by default, with working web code in `lib/src/web/` and assets in `web/`. Here's how to add a simple JSON route:

```
import 'dart:convert';

import 'package:serverpod/serverpod.dart';

class HelloRoute extends Route {
  @override
  Future<Result> handleCall(Session session, Request request) async {
    return Response.ok(
      body: Body.fromString(
        jsonEncode({'message': 'Hello from Serverpod!'}),
        mimeType: MimeType.json,
      ),
    );
  }
}
```



Register the route in your `server.dart` file before starting the server:

```
pod.webServer.addRoute>HelloRoute(), '/api/hello');
await pod.start();
```

Visit `http://localhost:8082/api/hello` to see your response. Port 8082 is the web server's default development port, configured per run mode in the [server configuration](#).

INFO

If your project was created with the "None" web server option, the first use of `pod.webServer` throws `Bad state: Web server is disabled`. To enable it, add the `webServer` section to your `config/<run mode>.yaml` files and register at least one route before `pod.start()`. With a configuration but no routes, the web server simply does not start.

Core concepts

Routes and handlers

A **route** is a destination in your web server that handles requests and generates responses. Routes extend the `Route` base class and implement the `handleCall()` method:

```
class ApiRoute extends Route {
  @override
  Future<Result> handleCall(Session session, Request request) async {
    // Your logic here
    return Response.ok();
  }
}
```

The `handleCall()` method receives:

- **Session** - Access to your database, logging, and authenticated user. The web server creates a `WebCallSession` for each request and closes it when the response is sent.
- **Request** - The HTTP request with headers, body, and URL information.

By default, a route answers GET requests only. Pass `methods:` to the constructor to accept others:

```
class FormRoute extends Route {
  FormRoute() : super(methods: {Method.get, Method.post});
  // ...
}
```

A request with a method the route does not accept gets an automatic `405 Method Not Allowed` response.

Response types

Each named `Response` constructor maps to an HTTP status code:

Constructor	Status
<code>Response.ok</code>	200
<code>Response.noContent</code>	204
<code>Response.movedPermanently</code>	301
<code>Response.found</code>	302
<code>Response.seeOther</code>	303
<code>Response.notModified</code>	304
<code>Response.badRequest</code>	400
<code>Response.unauthorized</code>	401
<code>Response.forbidden</code>	403
<code>Response.notFound</code>	404
<code>Response.contentTooLarge</code>	413
<code>Response.internalServerError</code>	500
<code>Response.notImplemented</code>	501

Status codes without a named constructor, such as `201 Created`, use the generic form: `Response(201, body: ...)`.

Adding routes

Routes are added with a path pattern:

```
// Exact path
pod.webServer.addRoute(UserRoute(), '/api/users');

// Serve a directory (tail matching is automatic)
pod.webServer.addRoute(StaticRoute.directory(Directory('web')), '/static/');
```

Paths can contain parameters and wildcards, and requests are matched by specificity rather than registration order. See [Routing](#) for the matching rules.

Built-in routes

Every web server automatically answers the health probe paths `/livez`, `/readyz`, and `/startupz`. A healthy server responds with an empty `200 OK`, and deployment platforms use these paths to check that the server is alive. These paths are reserved, so avoid registering your own routes on them.

When to use what

You want to serve	Route type	Page
REST APIs, webhooks, custom HTTP handlers	Your own <code>Route</code> subclass	Routing
Static assets: CSS, JavaScript, images	<code>StaticRoute</code>	Static files
Server-rendered HTML, with templates or Jaspr	<code>WidgetRoute</code> , or a route rendering Jaspr	Server-side HTML
A single-page app with client-side routing	<code>SpaRoute</code>	Single-page apps
Your Flutter app compiled for the web	<code>FlutterRoute</code>	Flutter web

Database access

The `Session` parameter gives you full access to your Serverpod database:

```
class UserRoute extends Route {
  @override
  Future<Result> handleCall(Session session, Request request) async {
    // Query database
    final users = await User.db.find(session);

    // Use logging
```

```
session.log('Retrieved ${users.length} users');

return Response.ok(
  body: Body.fromString(
    jsonEncode(users.map((u) => u.toJson()).toList()),
    mimeType: MediaType.json,
  ),
);
}
```

Going to production

When you deploy, the web server ships with the rest of your project. On [Serverpod Cloud](#), it is served through a CDN that honors the cache headers your routes set. See [Content delivery and caching](#) for how the two interact.

Next steps

- **Routing** - Match requests to handlers by method and URL pattern
- **Request data** - Access path parameters, query parameters, headers, and body
- **Web server middleware** - Intercept and transform requests and responses
- **Static files** - Serve static assets
- **Server-side HTML** - Render HTML on the server with templates or Jasper
- **Single-page apps** - Serve SPAs with client-side routing
- **Flutter web** - Serve Flutter web applications